

ThreadLocal之三

编写线程安全的代码

北京理工大学计算机学院
金旭亮

概述

“线程安全”，是多线程开发中经常遇到的一个术语。

“线程安全”的代码，是指在多线程环境下，使用者无须考虑线程同步问题（比如是否要加锁），可以跨线程直接调用的代码。

“线程安全”的代码，主要是通过内部封装线程同步逻辑，或者组合特定的多线程同步组件实现的。

“线程安全”的组件实例

ConcurrentHashMap是JDK中的一个“线程安全的”组件，它提供的方法，可以被跨线程安全地调用。

仔细看看ConcurrentHashMap的源码，不难发现，之所以这些方法可以安全地跨线程调用，它们内部都使用了**同步代码块**这一线程同步特性。

使用ThreadLocal，也能编写出“线程安全”的组件出来。

```
public class ConcurrentHashMap {  
    final V putVal(K key, V value, boolean onlyIfAbsent) {  
        synchronized(f) {  
            ...  
        }  
    }  
    final V replaceNode(Object key, V value, Object cv) {  
        synchronized(f) {  
            ...  
        }  
    }  
    public void clear() {  
        synchronized(f) {  
            ...  
        }  
    }  
}
```

ThreadLocal示例

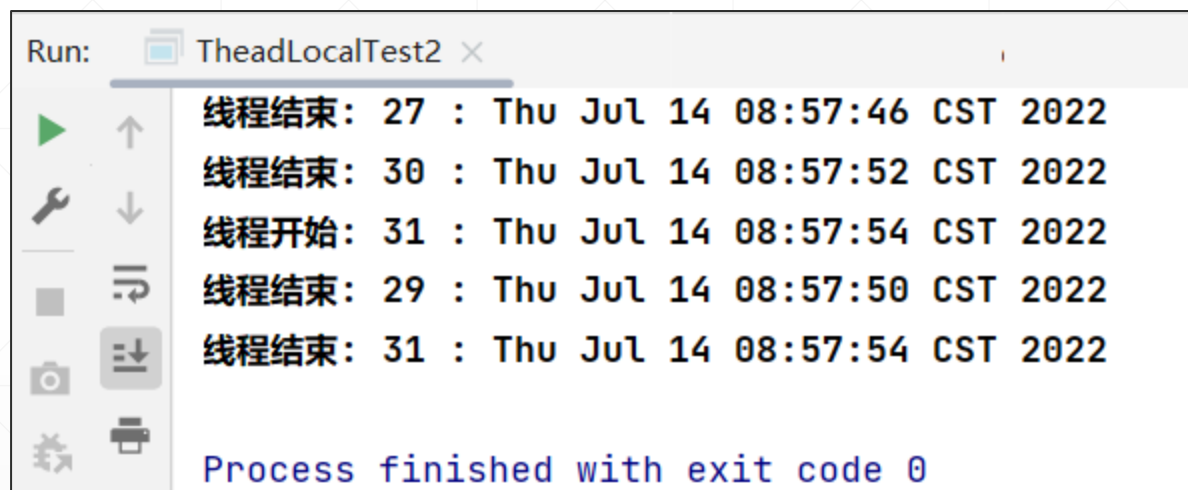
可以在线程函数中直接定义
ThreadLocal成员

```
class ThreadLocalFunc implements Runnable {  
    // 每个线程, 都有自己的startDate实例  
    private static ThreadLocal<Date> startDate =  
        ThreadLocal.withInitial(() -> new Date());  
    @Override  
    public void run() {  
        System.out.printf("线程开始: %s : %s\n",  
            Thread.currentThread().getId(), startDate.get());  
        try {  
            TimeUnit.SECONDS.sleep((int) Math rint(Math.random() * 10));  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.printf("线程结束: %s : %s\n",  
            Thread.currentThread().getId(), startDate.get());  
    }  
}
```

ThreadLocal示例

```
public class TheadLocalTest {  
    public static void main(String[] args) {  
        int processorCount = Runtime.getRuntime().availableProcessors();  
        System.out.println("本机CPU核数: " + processorCount);  
        // 创建多个线程  
        for (int i = 0; i < 2 * processorCount; i++) {  
            Thread thread = new Thread(new ThreadLocalFunc());  
            try {  
                TimeUnit.SECONDS.sleep(2);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            thread.start();  
        }  
    }  
}
```

运行截图



The screenshot shows a 'Run' window titled 'TheadLocalTest2'. On the left is a toolbar with icons for play, step up, step down, run and debug, camera, and a refresh icon. The main area contains the following log output:

```
线程结束: 27 : Thu Jul 14 08:57:46 CST 2022  
线程结束: 30 : Thu Jul 14 08:57:52 CST 2022  
线程开始: 31 : Thu Jul 14 08:57:54 CST 2022  
线程结束: 29 : Thu Jul 14 08:57:50 CST 2022  
线程结束: 31 : Thu Jul 14 08:57:54 CST 2022  
  
Process finished with exit code 0
```

可以看到，每个线程都保存了自己运行的开始时间和结束时间，彼此互不影响。

线程安全的对象

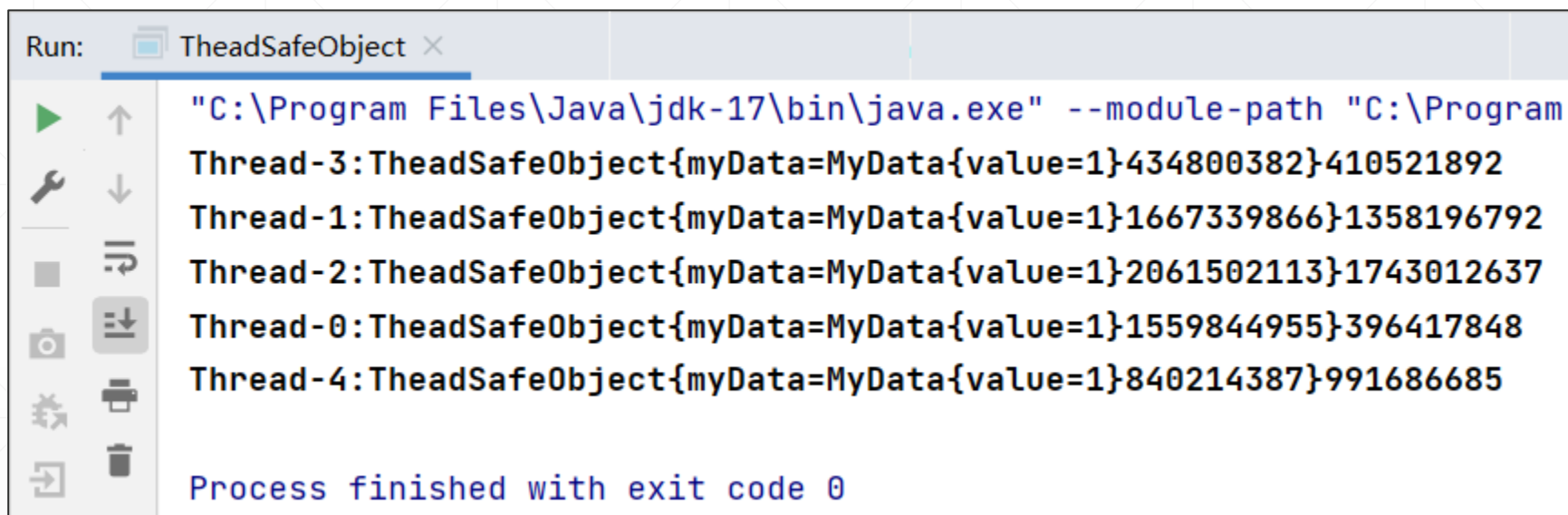
```
public class TheadSafeObject {  
    //内部包容的数据对象  
    private final MyData myData = new MyData();  
    public MyData getMyData() {...}  
    // 将被多线程同时访问的对象  
    private static final ThreadLocal<TheadSafeObject> objectstore  
        = new ThreadLocal<>();  
    // 构造函数私有, 不允许直接new它  
    private TheadSafeObject() {}  
    // 外界只能通过这个方法获取实例  
    public static TheadSafeObject getInstance() {  
        TheadSafeObject instance = objectstore.get();  
        if (instance == null) {  
            instance = new TheadSafeObject();  
            objectstore.set(instance);  
        }  
        return instance;  
    }  
    //当线程结束时, 应该注意释放掉内存  
    public static void disposeInstance() {  
        objectstore.remove();  
    }  
}
```

在需要多线程共享的对象内部，封装一个`ThreadLocal<T>`类型的字段，它引用一个对象。当一个线程调用`ThreadLocal.get()`方法获取这个对象的引用时，它将得到一个这个对象的“本地拷贝”并且独享它，从而无需锁定。

使用线程安全的对象

```
public static void main(String[] args) {  
    // 启动五个线程，运行相同的代码，看看是否能不用加锁而安全访问TheadSafeObject实例  
    for (int i = 0; i < 5; i++) {  
        new Thread(new Runnable() {  
            public void run() {  
                String threadname = Thread.currentThread().getName();  
                // 创建一个线程独享的对象  
                TheadSafeObject obj = TheadSafeObject.getInstance();  
                obj.getMyData().increase();  
                System.out.println(threadname + ":" + obj);  
                // 清理资源  
                TheadSafeObject.disposeInstance();  
            }  
        }).start();  
    }  
}
```


运行截图



```
Run: TheadSafeObject x
"C:\Program Files\Java\jdk-17\bin\java.exe" --module-path "C:\Program
Thread-3:TheadSafeObject{myData=MyData{value=1}434800382}410521892
Thread-1:TheadSafeObject{myData=MyData{value=1}1667339866}1358196792
Thread-2:TheadSafeObject{myData=MyData{value=1}2061502113}1743012637
Thread-0:TheadSafeObject{myData=MyData{value=1}1559844955}396417848
Thread-4:TheadSafeObject{myData=MyData{value=1}840214387}991686685

Process finished with exit code 0
```

从程序输出可以看到，每个线程都得到了独立的数据对象，可以单独对它进行存取。

小结



在多线程开发中，线程安全的对象可以被跨线程安全地访问，从而极大地简化了开发。



线程安全的代码，本质上就是将线程同步机制的相关代码封装在类的内部，本讲所介绍的使用`ThreadLocal`就是一种模式。



掌握编写线程安全代码的基本技能，是很实用的一种技能。